Part 2 – Simulation Report

1 Robot Setup

1.1 Robot Model

The robot was implemented using a custom URDF/Xacro model and successfully spawned in Gazebo. The model includes the main base, wheel links, IMU link, camera link, and camera optical frame.

- Robot description file: `my_robot.urdf.xacro`

1.2 TF Tree

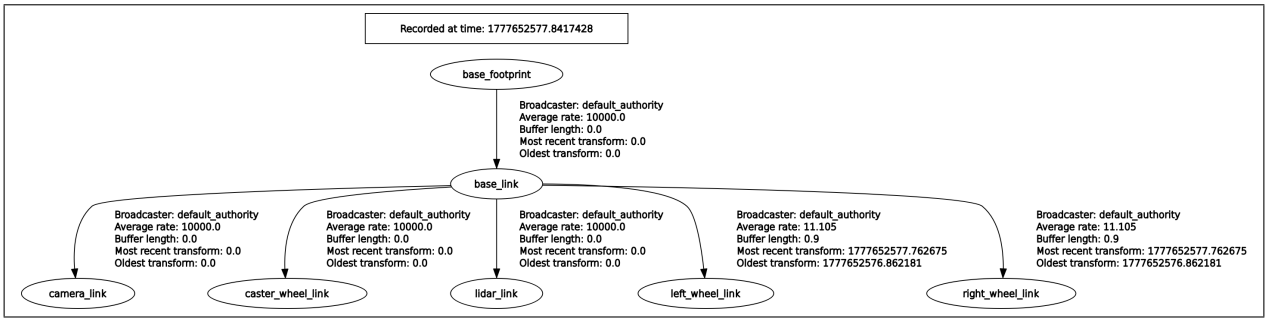

Figure 1: TF tree exported from `rqt_tf_tree`.

1.3 Frame Explanation

The robot follows the standard ROS coordinate convention: X forward, Y left, and Z upward.

- `base_footprint`: Ground projection of the robot base, used as the planar reference frame.
- `base_link`: Main body frame of the robot and parent frame for wheels and sensors.
- `camera_link`: Physical camera frame mounted at the front of the robot.
- `imu_link`: IMU mounting frame fixed to the robot base.
- `lidar_link`: Lidar mounted at the top of the robot.
- `left_wheel_link`: Left wheel frame connected to `base_link`.
- `right_wheel_link`: Right wheel frame connected to `base_link`.

2 Sensor Integration

2.1 IMU

The IMU was added using a Gazebo plugin and publishes data to:

```
/imu
```

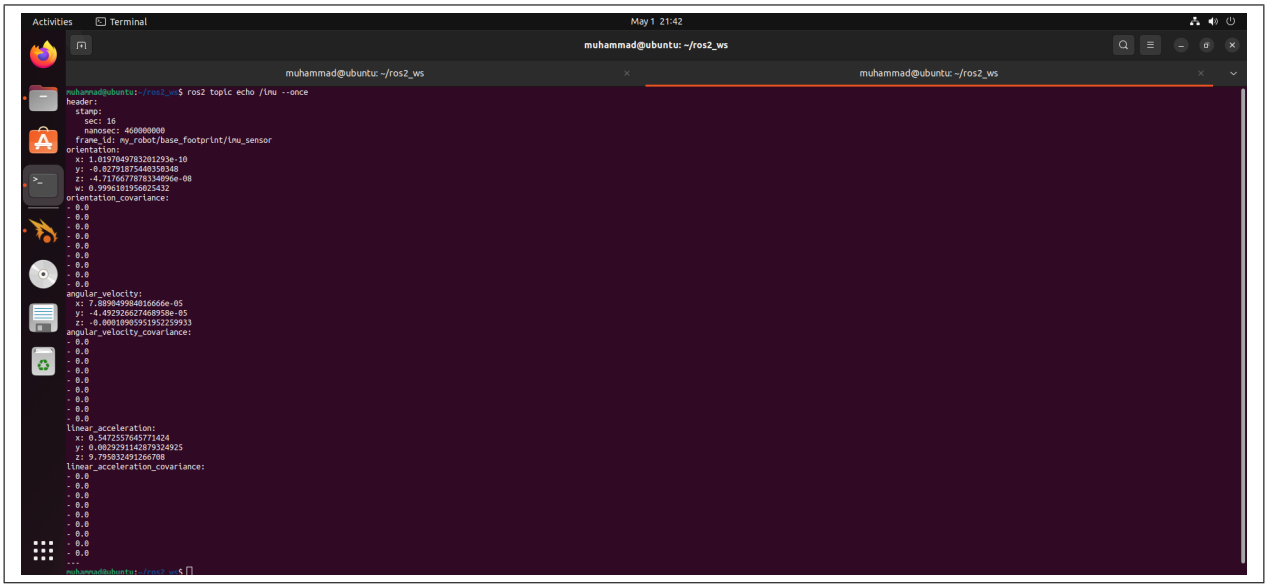

Figure 2: IMU plugin configured and publishing to a ROS 2 topic.

2.2 Camera

The camera was added using a Gazebo camera plugin and publishes image data to:

```
/image_raw
```

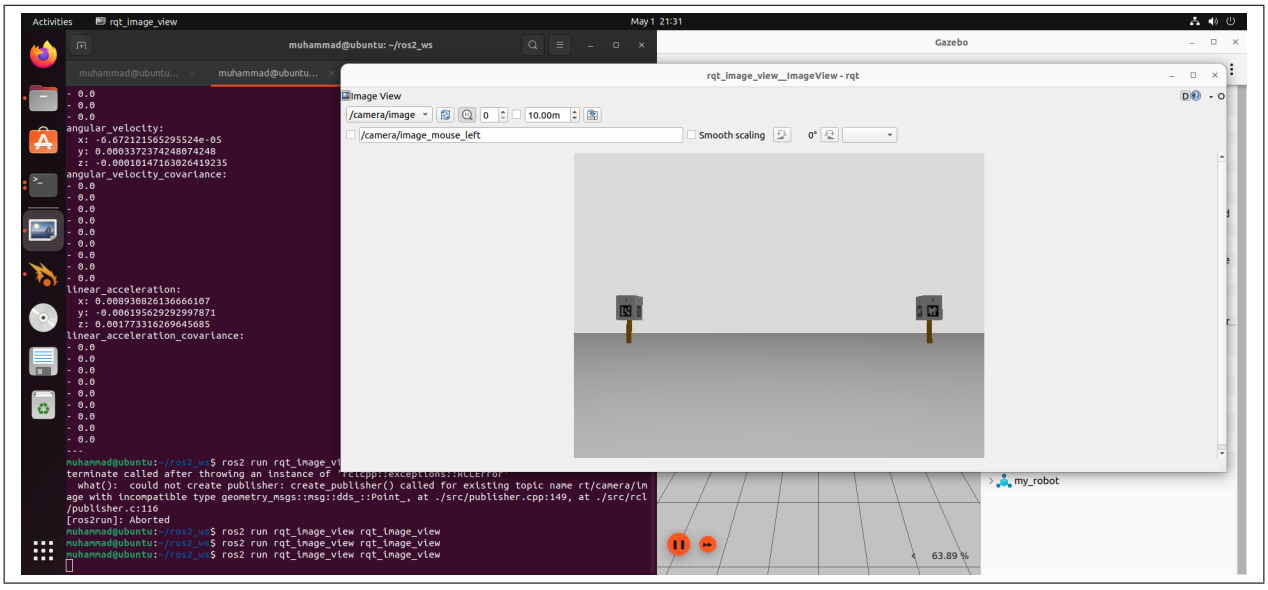

Figure 3: Camera plugin configured and publishing image data.

2.3 Sensor Placement

The IMU was placed close to the robot center to reduce the effect of rotational acceleration and vibration. The camera was mounted at the front of the robot so it can observe markers and obstacles ahead of the robot.

3 Sensor Validation

3.1 IMU Noise and Bias

The IMU topic was inspected using:

```
ros2 topic echo /imu
ros2 bag record /imu
rqt_plot /imu/linear_acceleration/x /imu/linear_acceleration/y /imu/linear_acceleration/z
rqt_plot /imu/angular_velocity/x /imu/angular_velocity/y /imu/angular_velocity/z
```

When the robot was stationary, angular velocity readings were close to zero, while linear acceleration showed the effect of gravity. The simulated IMU readings were mostly stable, with low noise and minimal bias depending on the plugin configuration.

3.2 Drift

The simulated IMU did not show significant long-term drift while the robot was stationary. Any small variations remained bounded and did not noticeably accumulate over time.

3.3 Simulation Fidelity

Compared to a real IMU, the Gazebo IMU is more ideal and less affected by temperature changes, mechanical vibration, calibration error, and long-term drift. It is useful for testing ROS 2 integration, but realistic noise and bias parameters are needed for a closer real-world comparison.

4 Environment Setup

4.1 Custom World

The Gazebo world file loads successfully and contains four ArUco marker poles arranged in a square pattern.

- World file: `poles_world.sdf`

4.2 Marker Coordinates

Marker Pole	X	Y	Z
Marker 1	2	2	0
Marker 2	-2	2	0
Marker 3	-2	-2	0
Marker 4	2	-2	0

Table 1: ArUco marker pole coordinates.

4.3 World View

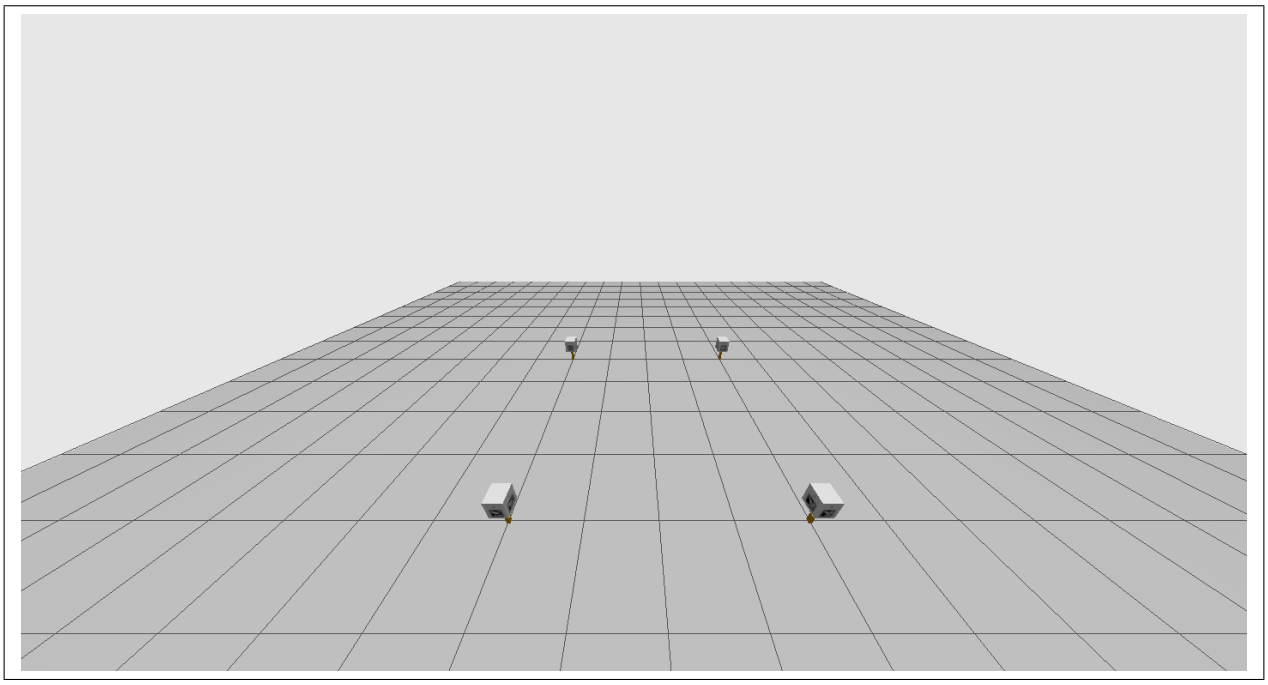

Figure 4: Gazebo world showing four ArUco marker poles arranged in a square pattern.

4.4 Startup Camera View

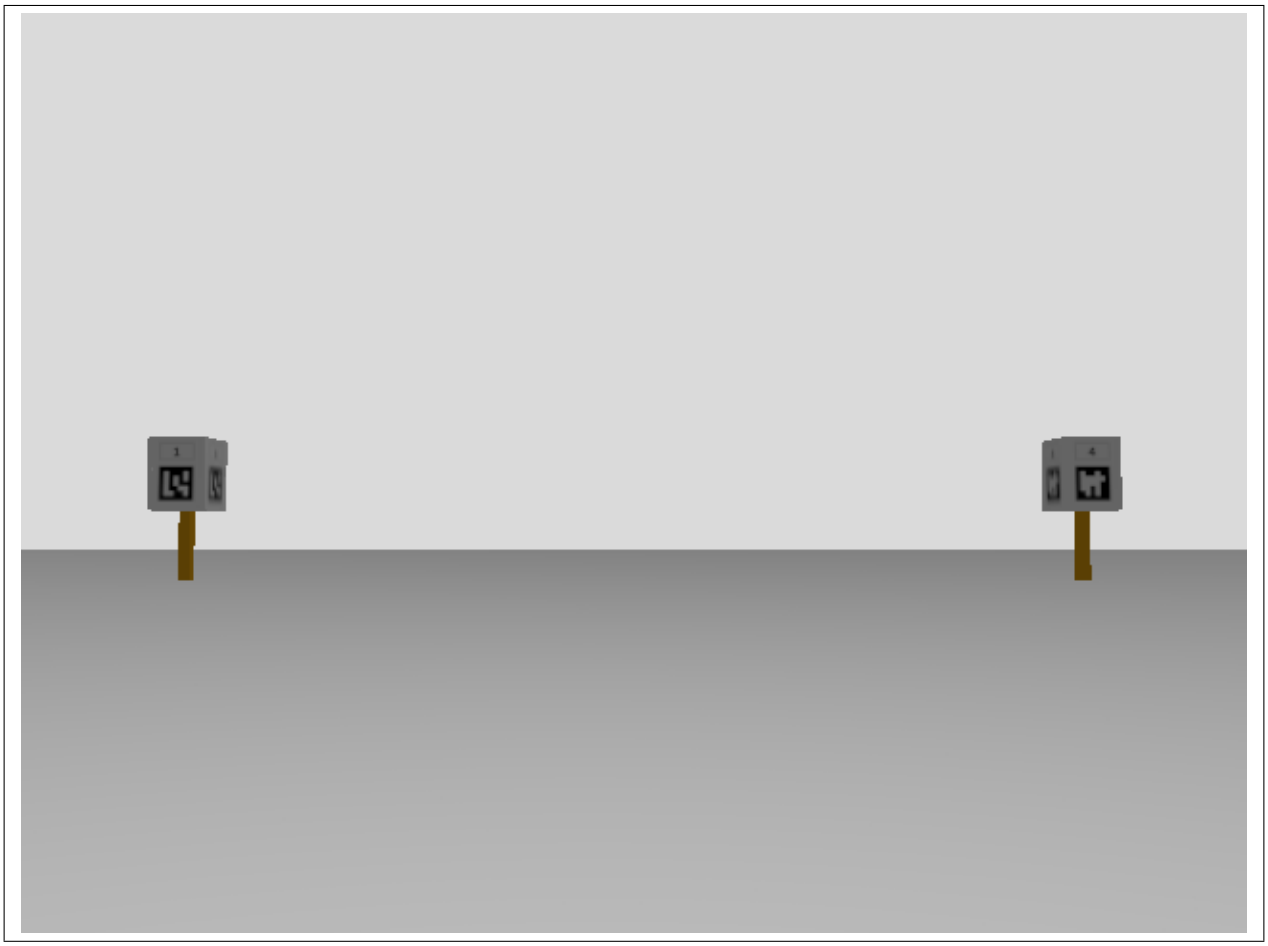

Figure 5: Robot camera view showing at least two visible ArUco markers at startup.

5 Perception Integration

5.1 Detection Node

The ArUco detection node was launched and connected to the robot camera topic. The node publishes detected marker pose data.

```
ros2 run aruco_tracking aruco_node
ros2 topic echo /perception/detected_aruco
```

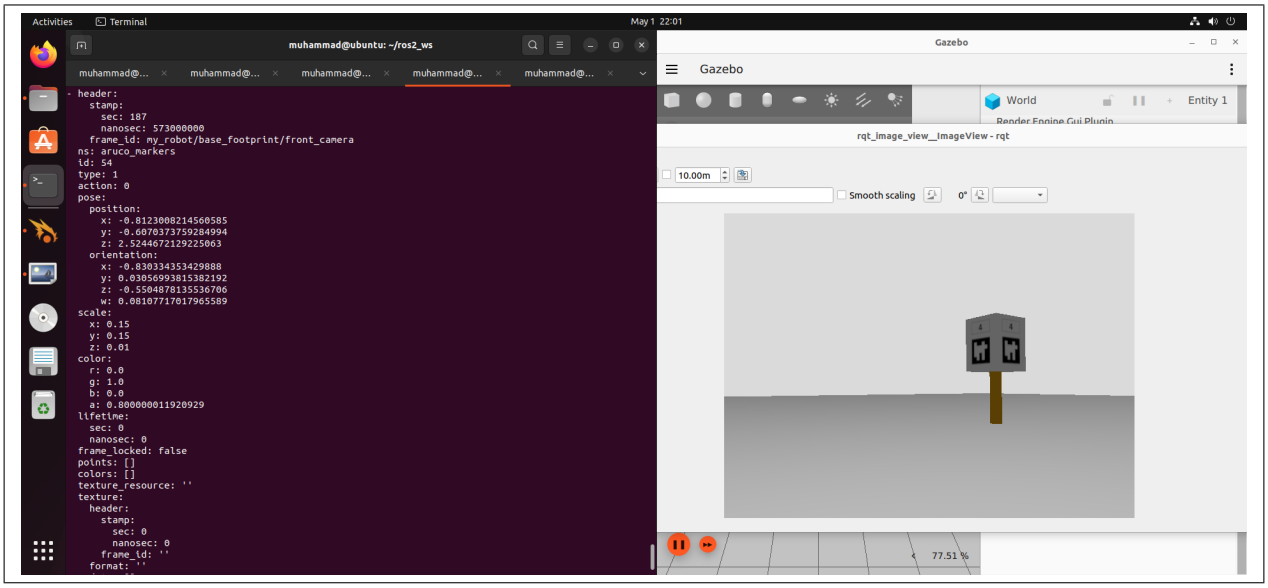

Figure 6: ArUco detection node running and publishing marker pose data.

5.2 Robot TF in RViz

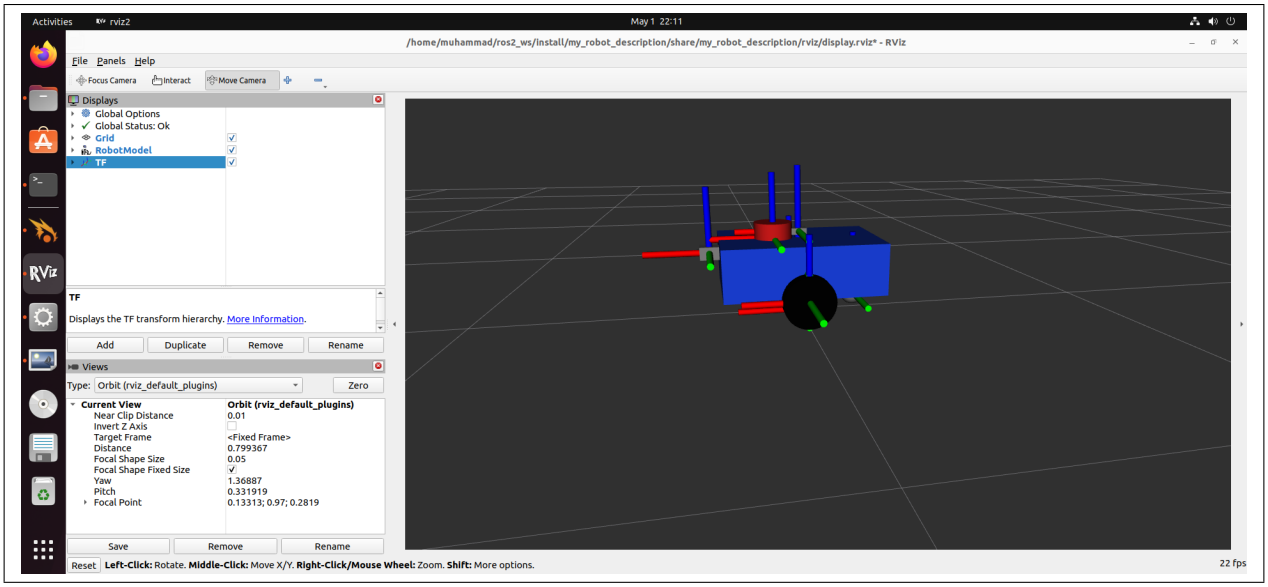

Figure 7: Robot TF frames visible in RViz.

5.3 Marker Frames in RViz

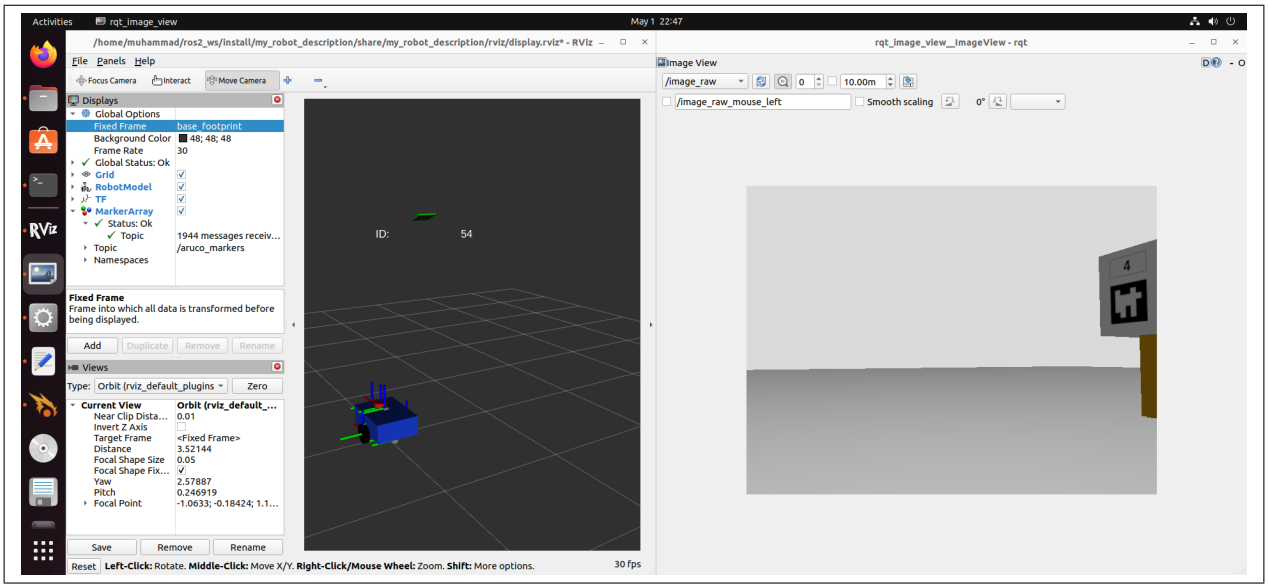

Figure 8: Detected ArUco marker frames overlaid correctly in RViz.

6 System Verification

6.1 Node Graph

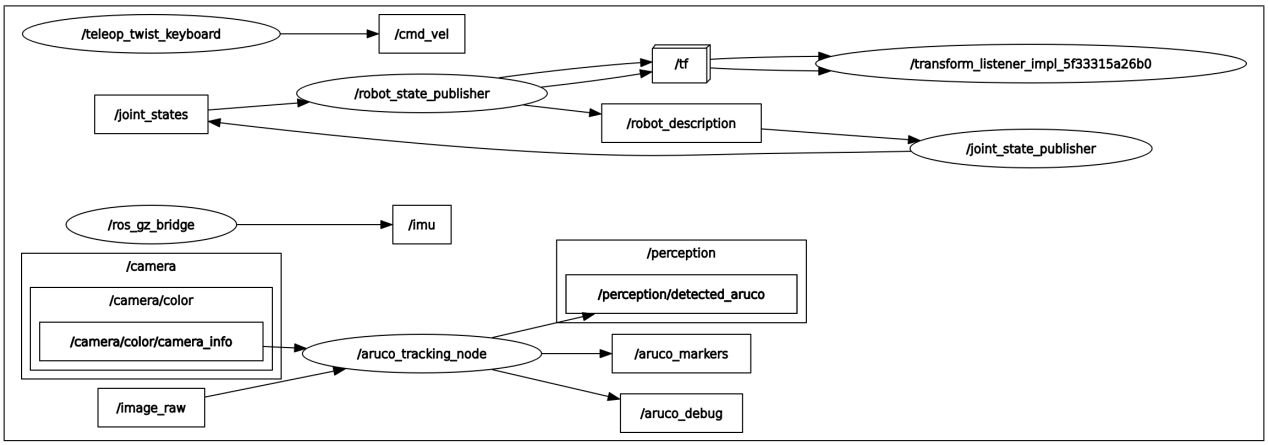

Figure 9: `rqt_graph` showing all nodes and topic connections.

6.2 TF Tree Export

- TF Tree file: `frames.pdf`

The TF tree was exported using:

```
ros2 run tf2_tools view_frames
```

6.3 Sensor-to-Perception Flow

```
ros2 topic hz /image_raw
ros2 topic hz /imu
ros2 topic echo /perception/detected_aruco --once
```

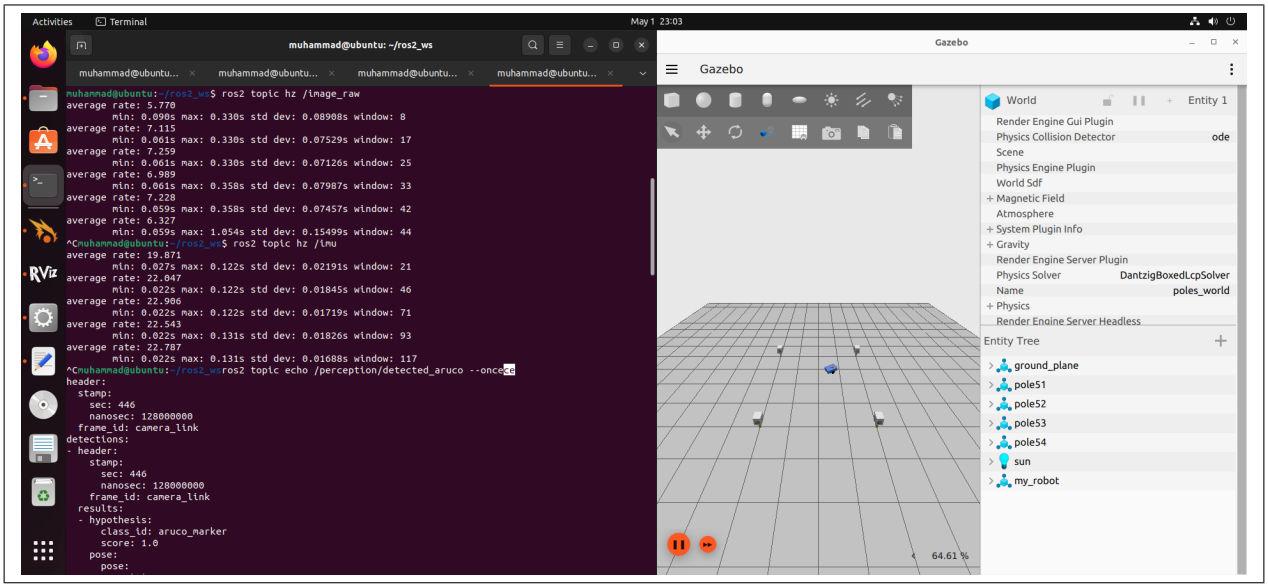

Figure 10: Evidence of sensor-to-perception data flow.

6.4 Anomalies and Fixes

Anomaly	Cause	Resolution
Duplicate detection of the same marker at different poses	Each pole uses the same ArUco marker on all four sides of the cube. When the camera views the pole from an angle, more than one side of the cube can be visible, causing the detector to identify the same marker ID multiple times with different estimated poses.	The issue was identified as a model/design artifact rather than a ROS 2 communication error. The detections were checked visually in RViz, and the marker pole design was considered when interpreting the detected poses. A possible improvement is to use unique marker IDs on each side or filter duplicate detections by marker ID and distance.

Table 2: System anomalies and resolutions.